

1. Что такое goroutine в Go?

- A) Обычный системный процесс операционной системы
- B) Легковесная единица выполнения, запускаемая ключевым словом go
- C) Файл конфигурации модуля
- D) Специальный тип ошибки

Ответ: B

Объяснение: Goroutine запускается через go и выполняется конкурентно под управлением рантайма Go.

2. Как правильно объявить переменную с автоматическим выводом типа внутри функции?

- A) var name := "Go"
- B) let name = "Go"
- C) name := "Go"
- D) const name := "Go"

Ответ: C

Объяснение: Короткое объявление := доступно внутри функций и выводит тип по значению справа.

3. Какое нулевое значение имеет переменная типа int?

- A) nil
- B) 0
- C) false
- D) пустая строка

Ответ: B

Объяснение: Для числовых типов нулевое значение равно 0.

4. Какие типы в Go могут использоваться как nil-значения?

- A) slice
- B) map
- C) channel
- D) pointer

Ответ: A,B,C,D

Объяснение: Срезы, карты, каналы и указатели могут иметь значение nil.

5. Как выполняются несколько defer-вызовов в одной функции?

- A) В случайном порядке
- B) В порядке объявления сверху вниз
- C) В обратном порядке, как стек LIFO
- D) Только первый defer будет выполнен

Ответ: C

Объяснение: Отложенные вызовы выполняются перед выходом из функции в обратном порядке добавления.

6. Что произойдет при чтении из закрытого канала, когда все отправленные значения уже прочитаны?

- A) Программа всегда упадет с panic
- B) Будет возвращено нулевое значение типа, а ok будет false
- C) Канал автоматически откроется снова
- D) Чтение заблокируется навсегда

Ответ: B

Объяснение: Форма value, ok := <-ch позволяет понять, что канал закрыт: ok будет false.

7. Какой метод должен иметь тип, чтобы реализовать стандартный интерфейс error?

- A) Error() string
- B) Message() string
- C) String() error
- D) Err() bool

Ответ: A

Объяснение: Интерфейс error содержит один метод: Error() string.

8. Как в Go сделать идентификатор экспортируемым из пакета?

- A) Добавить ключевое слово public
- B) Добавить ключевое слово export
- C) Начать имя с заглавной буквы
- D) Положить файл в папку public

Ответ: C

Объяснение: Имена, начинающиеся с заглавной буквы, экспортируются из пакета.

9. Для чего обычно используется package main?

- A) Только для тестовых файлов
- B) Для сборки исполняемой программы
- C) Только для приватных библиотек
- D) Для хранения зависимостей

Ответ: B

Объяснение: Пакет main с функцией main является точкой входа исполняемого приложения.

10. Какая команда создает новый Go-модуль?

- A) go mod init
- B) go init module
- C) go package create
- D) go build module

Ответ: A

Объяснение: Команда go mod init создает файл go.mod и задает имя модуля.

11. Что гарантирует range при обходе map?

- A) Порядок всегда совпадает с порядком добавления ключей
- B) Ключи всегда сортируются по алфавиту
- C) Сначала обходятся числовые ключи, потом строковые
- D) Порядок обхода map не гарантирован

Ответ: D

Объяснение: Порядок итерации по map в Go не должен использоваться как стабильный.

12. Чем slice отличается от array?

- A) Array не может хранить числа
- B) Длина array входит в тип, а slice является дескриптором участка массива
- C) Slice всегда копирует все данные при передаче в функцию
- D) Array всегда nil по умолчанию

Ответ: B

Объяснение: Массив имеет фиксированную длину как часть типа, а срез ссылается на базовый массив.

13. Что такое data race?

- A) Оптимизация компилятора для map
- B) Ситуация, когда программа использует слишком много памяти
- C) Ошибка импорта пакета
- D) Одновременный доступ к одной памяти из goroutine без синхронизации, если есть запись

Ответ: D

Объяснение: Гонка данных возникает при конкурентном доступе к общим данным без защиты, когда хотя бы один доступ записывает.

14. Какой стандартный пакет чаще всего используют для Mutex и WaitGroup?

- A) time
- B) fmt
- C) sync
- D) runtime/debug

Ответ: C

Объяснение: Пакет sync содержит Mutex, RWMutex, WaitGroup, Once и другие примитивы синхронизации.

15. Для чего обычно используют context.Context?

- A) Для отмены операций
- B) Для дедлайнов и таймаутов
- C) Для передачи request-scoped значений
- D) Для хранения глобальных паролей приложения

Ответ: A,B,C

Объяснение: Context нужен для отмены, дедлайнов и небольших значений в рамках запроса; глобальные секреты там хранить не следует.

16. Что делает оператор select?

- A) Ожидает одну из нескольких операций с каналами
- B) Выбирает поле структуры по имени
- C) Сортирует slice
- D) Заменяет if в любых условиях

Ответ: A

Объяснение: select позволяет ждать несколько операций send/receive по каналам и выполнить готовую ветку.

17. В чем разница между make и new?

- A) new работает только со строками
- B) make инициализирует slice, map или channel, а new возвращает указатель на нулевое значение
- C) make всегда возвращает указатель
- D) new запускает goroutine

Ответ: B

Объяснение: make применяют к slice, map и channel; new выделяет память и возвращает *T.

18. Как тип реализует интерфейс в Go?

- A) Только через ключевое слово implements
- B) Только если интерфейс находится в том же пакете
- C) Неявно, если у типа есть все методы интерфейса
- D) Только через наследование класса

Ответ: C

Объяснение: В Go реализация интерфейса структурная и не требует явного объявления implements.

19. Где recover может перехватить panic?

- A) В любой строке после panic
- B) Внутри отложенной defer-функции
- C) Только в функции main
- D) Только в init-функции

Ответ: B

Объяснение: recover работает, когда вызывается внутри deferred-функции во время раскрутки panic.

20. Для чего нужны struct tags вида json:"name"?

- A) Чтобы управлять именами полей при JSON-кодировании и декодировании
- B) Чтобы сделать поле приватным
- C) Чтобы ускорить garbage collector
- D) Чтобы запретить импорт пакета

Ответ: A

Объяснение: Пакет encoding/json читает теги и использует их для имен полей и дополнительных правил.

21. Для чего используется iota?

- A) Для чтения файлов
- B) Для запуска HTTP-сервера
- C) Для блокировки goroutine
- D) Для генерации последовательных значений в const-блоке

Ответ: D

Объяснение: iota удобно применять при объявлении перечислений через const.

22. Что верно про pointer receiver у метода?

- A) Он позволяет методу изменять значение receiver
- B) Он запрещает вызывать метод у переменной
- C) Он делает метод экспортируемым
- D) Он может уменьшить копирование больших структур

Ответ: A,D

Объяснение: Указательный receiver может менять объект и полезен, когда копировать значение дорого или нежелательно.

23. Какая команда запускает тесты Go?

- A) go check
- B) go test
- C) go verify
- D) go run tests

Ответ: B

Объяснение: Стандартная команда для тестов — go test.

24. Как ведет себя buffered channel, пока буфер не заполнен?

- A) Любая отправка всегда блокируется
- B) Он работает только с int
- C) Отправка может завершиться без немедленного receiver
- D) Он автоматически закрывается после первого значения

Ответ: C

Объяснение: Буферизованный канал может принять значения до своей capacity без синхронного получателя.

25. Как безопасно проверить наличие ключа в map?

- A) `value, ok := m[key]`
- B) `ok := exists(m, key)`
- C) `value := map.has(key)`
- D) `if key in m`

Ответ: A

Объяснение: Форма `value, ok := m[key]` показывает, найден ли ключ, даже если `value` равно нулевому значению.

26. Что означает тип `any` в современном Go?

- A) Только строковый тип
- B) Только числовой тип
- C) Псевдоним для `interface{}`
- D) Специальный тип канала

Ответ: C

Объяснение: `any` является `alias` для `interface{}` и может хранить значение любого типа.

27. Что может произойти при `append`, если `capacity` текущего `slice` недостаточна?

- A) Программа обязательно завершится `panic`
- B) Будет выделен новый базовый массив
- C) `Slice` станет `map`
- D) `append` вернет `nil` всегда

Ответ: B

Объяснение: `append` может создать новый `underlying array`, поэтому результат `append` нужно присваивать обратно.

28. Что верно про файлы с суффиксом `_test.go`?

- A) Они компилируются только на Windows
- B) Они запрещены в Go-модулях
- C) Они используются командой `go test` и не входят в обычную сборку приложения
- D) Они всегда должны быть в `package main`

Ответ: C

Объяснение: Файлы `_test.go` предназначены для тестов и обрабатываются командой `go test`.

29. Что делает garbage collector в Go?

- A) Автоматически освобождает недостижимые объекты в памяти
- B) Форматирует код
- C) Проверяет HTTP-запросы
- D) Удаляет неиспользуемые пакеты из go.mod

Ответ: A

Объяснение: GC освобождает память объектов, которые больше недостижимы из работающей программы.

30. Какие инструменты стандартной поставки Go помогают поддерживать качество кода?

- A) gofmt
- B) go test
- C) go vet
- D) photoshop

Ответ: A,B,C

Объяснение: gofmt форматирует код, go test запускает тесты, go vet ищет подозрительные конструкции.